

Prop Drilling and Context API

1. Recap: What Are Props?

Props are how a **parent** component passes data down to a **child** component.

```
function Greeting({ name }) {  
  return <p>Hello, {name}!</p>;  
}  
  
function App() {  
  return <Greeting name="Alice" />;  
}
```

App owns the value "Alice" and hands it directly to Greeting, which uses it right away. Simple. One hop. No problem yet.

2. The Real-World Analogy

Imagine you want to send a birthday gift to your friend who lives 4 houses down the street.

Instead of walking it there yourself, you hand it to your **next-door neighbor**, and say: *"Please give this to the next house, and tell them to keep passing it along until it reaches house #5."*

Neighbor 1 doesn't want the gift. Neighbor 2 doesn't want it either. Neighbor 3 doesn't either. **None of them care about the gift — they're just middlemen forced to carry it because it's the only path available.**

That's prop drilling: passing a value through components that don't need it, just so it can reach one that does.

3. What Is Prop Drilling? (Formal Definition)

Prop drilling is when you pass data through several layers of components — most of which don't use that data themselves — just to get it to a deeply nested component that actually needs it.

It's not a bug. It's not broken code. It **works fine**. The problem is what it costs you as your app grows — covered in Section 5.

4. Watch the Pain Happen

Scenario: A logged-in user's name needs to be displayed deep inside a dashboard.

```
App
├── Dashboard
│   └── Sidebar
│       └── Profile
│           └── Avatar    <-- only THIS component actually needs the username
ame

// Avatar.jsx – the ONLY component that actually uses "username"
function Avatar({ username }) {
  return <p>👤 {username}</p>;
}

// Profile.jsx – doesn't use "username" itself, just passes it along
function Profile({ username }) {
  return (
    <div>
      <h3>Profile</h3>
      <Avatar username={username} />
    </div>
  );
}

// Sidebar.jsx – doesn't use "username" itself, just passes it along
function Sidebar({ username }) {
  return (
    <div>
      <h3>Sidebar</h3>
      <Profile username={username} />
    </div>
  );
}

// Dashboard.jsx – doesn't use "username" itself, just passes it along
function Dashboard({ username }) {
  return (
    <div>
      <h3>Dashboard</h3>
      <Sidebar username={username} />
    </div>
  );
}

// App.jsx – the ONLY component that actually OWNS "username"
function App() {
  const username = "Alice";
  return <Dashboard username={username} />;
}
```

```
}  
  
export default App;
```

Look closely at Dashboard, Sidebar, and Profile. **None of them do anything with username except immediately forward it.** They're the neighbors from the analogy — unwilling middlemen.

5. Pain Points — Highlighted

● Pain Point 1: Unrelated components are forced to know about data they don't care about

Sidebar has nothing to do with usernames. Yet its function signature is now permanently cluttered with `{ username }` just because something *below* it needs it.

● Pain Point 2: Adding a new piece of data means editing every file in the chain

Say you now also need to pass the user's email down to Avatar. You must edit **all four files** — App, Dashboard, Sidebar, Profile — even though three of them will never use email either.

```
- function Sidebar({ username }) {  
+ function Sidebar({ username, email }) {  
    ...  
-   <Profile username={username} />  
+   <Profile username={username} email={email} />
```

Repeat that same edit in Dashboard and Profile too. One new piece of data = four files touched.

● Pain Point 3: Renaming a prop breaks the entire chain

Decide to rename username to userName? You now have to find and fix it in **every single middle component**, or the chain silently breaks.

● Pain Point 4: It gets exponentially worse as the app grows

Two levels of drilling is mildly annoying. Real apps often have 6, 8, 10 levels of nested components. Multiply that by every piece of shared data (user info, theme, language, cart items...) and the middle components become unreadable prop-forwarding machinery.

● Pain Point 5: It's hard to trace where data actually comes from

If you're a new developer opening Avatar.jsx for the first time, you see username show up — but to find out *where* it originally came from, you must manually walk backward through 3 other files. This makes debugging and onboarding slower.

6. Mini Exercise

Copy the code from Section 4 into a project. Then:

1. Add a role prop (e.g. "admin") that Avatar should also display.
2. Wire it through App → Dashboard → Sidebar → Profile → Avatar.
3. Count how many files you had to touch to add **one** new value.

That number is the pain, made concrete.

7. conclusion

Prop drilling isn't wrong — it's just what happens by default when React only gives you one tool (props) for passing data down. It becomes a real problem once:

- The nesting gets deep (4+ levels), **or**
- Many unrelated components sit in the middle of the chain, **or**
- The data needs to reach many different branches of the tree, not just one path

In the next slide, we'll cover two ways to fix this:

- **Lifting state up** — a quick fix for small cases
- **Context API** — React's built-in solution for deeply nested data, so components like Sidebar and Profile never have to touch username at all

Prop drilling is passing data through components that don't need it, just to reach one that does — and the deeper your app grows, the more that "just passing it along" costs you.

8. Tasks & Projects

Task 1 — Count the Cost

Using the code from Section 4: 1. Add a new email prop and wire it through all four components down to Avatar. 2. Write down exactly how many files you had to open and how many lines you had to touch. 3. Answer in one sentence: what would happen if you had 10 pieces of data instead of 2?

Task 2 — Build It From Scratch (

Build a small **"Settings Panel"** app with this exact structure:

```
App
├── SettingsPage
│   └── SettingsSection
│       ├── ThemeToggle  <-- needs a `theme` prop ("light" / "dark")
│       └── ThemeLabel   <-- needs the same `theme` prop
```

Requirements: - App owns a theme state value ("light" or "dark"), initialized with useState. - theme must be drilled through SettingsPage → SettingsSection down to both ThemeToggle and ThemeLabel. - ThemeToggle should have a button that logs the current theme to the console when clicked (no need to actually switch it yet — this task is about drilling, not switching logic). - ThemeLabel should display Current theme: light or Current theme: dark.

Reflection questions to answer: - Which components in your tree don't actually use theme themselves, and just forward it? - If you added a second value (e.g. fontSize), how many extra lines would you need across your files?

Task 3 — Break It on Purpose

Take your Task 2 project and **rename** the theme prop to appTheme in App.jsx only — but leave every other file using the old name theme. - Run the app. What error or bug do you get? - Where does the mismatch actually surface — in the file where you made the change, or somewhere else entirely? - Write 2–3 sentences on why this makes debugging harder in a real, larger codebase.

Mini-Project — “Team Directory” App

Build a small team directory app with this component tree:



- App owns an array of employee objects, e.g.

```
const employees = [
  { name: "Alice", manager: "Grace" },
  { name: "Bob", manager: "Grace" },
  { name: "Carol", manager: "Frank" },
];
```

- The manager name must reach TeamMember by drilling through Company, Department, and TeamList — none of which use it directly.
- Render a list of TeamMember components, each showing: Alice (reports to Grace).

Deliverable: submit the code plus a short write-up (half a page) answering: 1. List every component that had to “carry” the manager prop without using it. 2. If the company added a new field like department or email for every employee, how many files would need to change, and why? 3. In your own words, what specific problem would Context API or lifting state up solve here?

This mini-project is designed to be revisited in the next topic — after covering Context API, we will refactor this exact same app and compare the before/after directly.

9. Part 2: Context API — Full Walkthrough

Everything above was about *feeling* the prop drilling problem. This part is entirely about the fix: the Context API, covered step by step so nothing is assumed.

9.1 What Is Context, in Plain Language?

Go back to the neighbor analogy from Section 2. Prop drilling was: hand the gift to Neighbor 1, who hands it to Neighbor 2, who hands it to Neighbor 3, who finally hands it to your friend at house #5.

Context is a delivery truck instead. You put the gift in a truck parked at the top of the street, and give your friend at house #5 a key to walk up and grab it directly — no neighbors involved, no forwarding, no middlemen.

That's the whole idea: **Context lets far-apart components share data without every component in between having to touch it.**

9.2 The Three-Step Recipe

Every single use of Context — no matter how big or small the app — follows the same three steps, in the same order:

1. **Create the box.** `const SomeContext = createContext();` — this makes an empty container that will hold your shared value.
2. **Provide a value into the box.** Wrap the part of your app that needs access in `<SomeContext.Provider value={...}>`. Whatever you put in `value` becomes available to everything inside.
3. **Consume the value where it's needed.** Inside any component nested under the `Provider` — no matter how deep — call `useContext(SomeContext)` to pull the value out directly.

Memorize the order: **Create → Provide → Consume**. Every Context example you'll ever see follows this exact sequence.

9.3 Step-by-Step: Building One From Scratch

Step 1 — Create the box.

```
import { createContext } from "react";  
  
const ThemeContext = createContext();
```

At this point the box exists, but it's empty. Nothing is using it yet.

Step 2 — Provide a value.

```
function App() {
  const theme = "dark";
  return (
    <ThemeContext.Provider value={theme}>
      <Dashboard />
    </ThemeContext.Provider>
  );
}
```

Now every component rendered inside `<Dashboard />` — no matter how many layers deep — has access to "dark" through this box, automatically.

Step 3 — Consume the value.

```
import { useContext } from "react";

function ThemeLabel() {
  const theme = useContext(ThemeContext);
  return <p>Current theme: {theme}</p>;
}
```

ThemeLabel can sit anywhere inside Dashboard — 1 level deep or 10 — and this code doesn't change. That's the entire point.

9.4 The Part Most Tutorials Skip: Updating the Value, Not Just Reading It

So far, the box only ever holds a fixed value. But real apps need components deep in the tree to **change** shared data too — like a button that toggles the theme. Here's the procedure for that.

The trick: put both the value AND the function that changes it into the box together.

```
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext();

function App() {
  const [theme, setTheme] = useState("light");

  return (
    // Pack both the value and the "changer" into one object
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Dashboard />
    </ThemeContext.Provider>
  );
}

function ThemeToggleButton() {
  const { theme, setTheme } = useContext(ThemeContext);
```

```

function handleClick() {
  setTheme(theme === "light" ? "dark" : "light");
}

return <button onClick={handleClick}>Switch to {theme === "light" ? "dark"
: "light"} mode</button>;
}

function ThemeLabel() {
  const { theme } = useContext(ThemeContext);
  return <p>Current theme: {theme}</p>;
}

```

Walk through: 1. App still owns the real state with `useState`, exactly like before Context was introduced. 2. Instead of putting just `theme` into the Provider's value, it puts an **object** containing both `theme` (the data) and `setTheme` (the function that changes it). 3. `ThemeToggleButton`, buried anywhere inside `Dashboard`, pulls out `setTheme` and calls it directly — updating state that lives all the way up in `App` — without a single prop being passed anywhere. 4. Because `theme` lives in `App`'s `useState`, updating it there causes `App` to re-render, which updates the value flowing into the Provider, which every consumer (like `ThemeLabel`) automatically picks up on its next render.

reading from Context and updating Context both go through the same box — you just have to remember to put the setter function in the box alongside the value.

9.5 Multiple Contexts

A common beginner mistake: cramming every piece of shared state (`theme`, `user` info, `cart`, `language`...) into one single giant context. Instead, create a **separate context for each unrelated concern**:

```

const ThemeContext = createContext();
const UserContext = createContext();

function App() {
  const [theme, setTheme] = useState("light");
  const [user, setUser] = useState({ name: "Alice" });

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <UserContext.Provider value={{ user, setUser }}>
        <Dashboard />
      </UserContext.Provider>
    </ThemeContext.Provider>
  );
}

```

Any component can now pick and choose: `useContext(ThemeContext)` if it only cares about theme, `useContext(UserContext)` if it only cares about the user — without being forced to also carry the other.

9.6 `useReducer` + Context (For When State Gets Complex)

If your shared state has several related pieces that update together — like a shopping cart with `addItem`, `removeItem`, and `clearCart` — a single `setSomething` function starts to feel messy. `useReducer` organizes this into one place:

```
import { createContext, useContext, useReducer } from "react";

const CartContext = createContext();

function cartReducer(state, action) {
  switch (action.type) {
    case "add":
      return [...state, action.item];
    case "clear":
      return [];
    default:
      return state;
  }
}

function App() {
  const [cart, dispatch] = useReducer(cartReducer, []);

  return (
    <CartContext.Provider value={{ cart, dispatch }}>
      <Dashboard />
    </CartContext.Provider>
  );
}

function AddToCartButton({ item }) {
  const { dispatch } = useContext(CartContext);
  return <button onClick={() => dispatch({ type: "add", item })}>Add to cart</button>;
}
```

Layman explanation: instead of many separate `setX` functions scattered around, `dispatch` sends a single labeled instruction (“add this”, “clear everything”) to one function (`cartReducer`) that decides how state should change. Not required for simple cases — only reach for this once plain `useState` starts feeling cluttered.

9.7 The One Real Downside: Context Re-renders Everything That Reads It

Rule: when a Context's value changes, *every* component that calls `useContext()` on it re-renders — even ones that only care about part of the value.

Example: if `ThemeContext` holds `{ theme, setTheme }` and `theme` changes, any component reading that context re-renders, even one that only ever displays a totally unrelated piece you later added to that same object.

The fix, procedurally: 1. Keep contexts small and split by concern (Section 9.5) — don't lump unrelated data into one Provider. 2. For state that changes very frequently (e.g. mouse position, scroll offset), Context is usually the wrong tool — that's a sign a dedicated state management library (covered in a future topic) is a better fit.

10. The Avatar Problem, Refactored

Compare this directly against Section 4. Same result on screen — Avatar displays username — but watch what disappears.

```
import { createContext, useContext, useState } from "react";

// 1. Create the "box" that will hold the shared value
const UserContext = createContext();

// Avatar.jsx - reaches directly into the box, no prop needed
function Avatar() {
  const username = useContext(UserContext);
  return <p>👤 {username}</p>;
}

// Profile.jsx - no longer mentions "username" anywhere
function Profile() {
  return (
    <div>
      <h3>Profile</h3>
      <Avatar />
    </div>
  );
}

// Sidebar.jsx - no longer mentions "username" anywhere
function Sidebar() {
  return (
    <div>
      <h3>Sidebar</h3>
      <Profile />
    </div>
  );
}
```

```

}

// Dashboard.jsx – no longer mentions "username" anywhere
function Dashboard() {
  return (
    <div>
      <h3>Dashboard</h3>
      <Sidebar />
    </div>
  );
}

// App.jsx – still owns the state, now "provides" it into the box
function App() {
  const [username] = useState("Alice");
  return (
    <UserContext.Provider value={username}>
      <Dashboard />
    </UserContext.Provider>
  );
}

export default App;

```

Dashboard, Sidebar, and Profile — the three “unwilling neighbors” from the analogy no longer receive or mention `username` at all. Their function signatures went from `({ username })` back to `()`. - App didn’t lose the state — it still owns `username` via `useState`. It just also wraps its children in `<UserContext.Provider value={username}>`, making that `value` reachable from anywhere inside. - Avatar is the only other component that changed — it swapped `props.username` for `useContext(UserContext)`. One line.

Task 4 — Refactor Your Own Project

Take your Task 2 “Settings Panel” project (or the Team Directory mini-project) and refactor it using `createContext` + `useContext`, following the pattern above.

Then answer directly: 1. Which files lost a prop entirely? 2. If you now add a second shared value (e.g. `fontSize` or `department`), how many files do you need to touch this time, compared to your earlier count in Task 1 or the mini-project write-up? 3. Is there any downside you can think of to putting everything in Context? (Hint: what happens if every piece of state in an app lives in one giant context?)

Task 5 — Make It Updatable

Extend your Task 4 refactor: add a button somewhere deep in the tree (e.g. inside `ThemeToggle` or `TeamMember`) that changes the shared value, following the pattern in Section 9.4 (pack the value **and** its setter into the Provider). Confirm that clicking the button updates every component reading that context, without passing a single new prop.

Task 6 — Split the Context

Add a second, unrelated piece of shared state to your project (e.g. language alongside theme). Instead of adding it to your existing context's object, create a **second, separate context** for it, following Section 9.5. Explain in 2–3 sentences why keeping them separate is better than combining them into one.

Mini-Project 2 — “Shopping Cart” with useReducer + Context

Build a small app with:

- A CartContext created with createContext
- Cart state managed with useReducer, supporting at least add, remove, and clear actions
- A ProductList component (renders a few hardcoded products with “Add to cart” buttons) nested several components deep
- A CartSummary component (shows item count and a “Clear cart” button) nested in a completely different branch of the tree

Deliverable: the code, plus 3–4 sentences explaining why useReducer was a better fit here than several separate useState + setter functions in the context.

Coming in a future topic: Zustand and Redux — dedicated state management tools that solve the re-render cost from Section 9.7 and remove the “wrap everything in Providers” ceremony entirely. This lecture intentionally stops at Context so you feel exactly which of its limitations those tools are built to solve.

Compare this directly against Section 4. Same result on screen — Avatar displays username — but watch what disappears.

```
import { createContext, useContext, useState } from "react";

// 1. Create the "box" that will hold the shared value
const UserContext = createContext();

// Avatar.jsx – reaches directly into the box, no prop needed
function Avatar() {
  const username = useContext(UserContext);
  return <p>👤 {username}</p>;
}

// Profile.jsx – no longer mentions "username" anywhere
function Profile() {
  return (
    <div>
      <h3>Profile</h3>
      <Avatar />
    </div>
  );
}

// Sidebar.jsx – no longer mentions "username" anywhere
function Sidebar() {
```

```

    return (
      <div>
        <h3>Sidebar</h3>
        <Profile />
      </div>
    );
  }

// Dashboard.jsx – no longer mentions "username" anywhere
function Dashboard() {
  return (
    <div>
      <h3>Dashboard</h3>
      <Sidebar />
    </div>
  );
}

// App.jsx – still owns the state, now "provides" it into the box
function App() {
  const [username] = useState("Alice");
  return (
    <UserContext.Provider value={username}>
      <Dashboard />
    </UserContext.Provider>
  );
}

export default App;

```

Dashboard, Sidebar, and Profile — the three “unwilling neighbors” from the analogy no longer receive or mention username at all. Their function signatures went from ({ username }) back to (). - App didn’t lose the state — it still owns username via useState. It just also wraps its children in <UserContext.Provider value={username}>, making that value reachable from anywhere inside. - Avatar is the only other component that changed — it swapped props.username for useContext(UserContext). One line.

Task 4 — Refactor Your Own Project

Take your Task 2 “Settings Panel” project (or the Team Directory mini-project) and refactor it using createContext + useContext, following the pattern above.

Then answer directly: 1. Which files lost a prop entirely? 2. If you now add a second shared value (e.g. fontSize or department), how many files do you need to touch this time, compared to your earlier count in Task 1 or the mini-project write-up? 3. Is there any downside you can think of to putting everything in Context? (Hint: what happens if every piece of state in an app lives in one giant context?)